

Descriptif de la Stack Technique

Plateforme de Gestion Documentaire Filière Santé Numérique (FSN)

Version	1.0
Date	15 avril 2026
Statut	Document de référence
Classification	Interne

Ce document décrit l'architecture technique et les choix technologiques retenus pour la plateforme de gestion documentaire de la FSN.

Il fait suite au compte rendu de visioconférence du 15 avril 2026 avec Michaël Harbouche (Agile Solutions).

Historique des versions

Version	Date	Auteur	Description
1.0	15/04/2026	Moss'Ab Mirande-Ney	Rédaction initiale suite à la visioconférence avec M. Harbouche

Table des matières

Historique des versions	1
1 Présentation générale	3
1.1 Contexte	3
1.2 Périmètre du document	3
2 Framework applicatif	3
2.1 Next.js — Framework principal	3
2.2 React et interface utilisateur	4
2.3 Styles	4
3 Base de données	4
3.1 ORM — Prisma	4
3.2 Base de données SQLite	4
3.3 Modèle de données	5
4 Authentification et sécurité	5
4.1 NextAuth.js	5
4.2 Mécanismes de sécurité	5
5 Stockage des fichiers	6
5.1 Principe de stockage	6
5.2 Types de fichiers acceptés	6
6 Infrastructure et hébergement	6
6.1 Système d'exploitation — Debian 11	6
6.2 Hébergeur — OVH (VPS)	6
6.3 Architecture serveur	7
6.4 Spécifications serveur recommandées	7
6.5 Absence de conteneurisation	7
7 Outils de développement	7
7.1 Chaîne de build	8
7.2 Scripts disponibles	8
7.3 Variables d'environnement	8
8 Perspectives — Phase 2 : Intelligence Artificielle	8
8.1 Intégration IA existante	8
8.2 Considérations sur les coûts d'infrastructure IA	8
9 Synthèse	9

1. Présentation générale

1.1. Contexte

Ce document a été rédigé à la suite de la visioconférence du 15 avril 2026 entre l'équipe de développement de la Junior Conseil UTT et Michaël Harbouche (Agile Solutions), architecte consultant du projet.

Il a pour objectif de fournir une description précise et complète de la stack technique retenue pour le développement de la plateforme de gestion documentaire de la Filière Santé Numérique (FSN).

1.2. Périmètre du document

Ce descriptif couvre :

- Le framework applicatif et les technologies front-end et back-end
- Le système de gestion de base de données
- Le système d'authentification et de sécurité
- L'infrastructure d'hébergement et le système d'exploitation
- Les outils de développement et la chaîne de build
- Les perspectives d'évolution (phase 2 — IA)

2. Framework applicatif

2.1. Next.js — Framework principal

L'application repose sur **Next.js 16.2.1**, un framework React full-stack permettant de gérer à la fois le rendu côté serveur (SSR), les routes API, et le rendu côté client au sein d'un seul et même projet.

Composant	Version
Next.js	16.2.1
React	19.2.4
React DOM	19.2.4
TypeScript	5.9.3

Choix justifiés :

- Architecture *App Router* pour un routage moderne et des Server Components performants
- Routes API intégrées (`/api/*`) évitant la nécessité d'un back-end distinct
- TypeScript en mode strict pour la robustesse et la maintenabilité du code

- Limite de taille des Server Actions portée à **52 Mo** pour supporter les uploads de documents volumineux

2.2. React et interface utilisateur

L'interface est construite sur **React 19.2.4** avec les bibliothèques suivantes :

Bibliothèque	Version	Rôle
Radix UI (11 composants)	divers	Composants accessibles (dialog, select, tabs...)
Framer Motion	12.38.0	Animations d'interface
Lucide React	1.7.0	Icônes
Sonner	2.0.7	Notifications <i>toast</i>
react-dropzone	15.0.0	Zone de dépôt pour l'upload de fichiers

2.3. Styles

Outil	Version	Rôle
Tailwind CSS	4.2.2	Framework CSS utilitaire (nouvelle API v4)
@tailwindcss/postcss	4.2.2	Plugin PostCSS pour Tailwind v4
class-variance-authority	0.7.1	Gestion des variantes de composants
clsx	2.1.1	Composition conditionnelle de classes
tailwind-merge	3.5.0	Fusion intelligente de classes Tailwind

3. Base de données

3.1. ORM — Prisma

L'accès aux données est géré via **Prisma 7.5.0**, un ORM (Object-Relational Mapper) TypeScript offrant une couche d'abstraction sur la base de données avec typage fort, migrations automatisées et interface d'administration.

3.2. Base de données SQLite

Conformément aux préconisations issues de la visioconférence, la base de données retenue est **SQLite**.

Composant	Version	Rôle
Prisma	7.5.0	ORM et gestion des migrations
@prisma/adapters-better-sqlite3	7.5.0	Adaptateur SQLite pour Prisma
better-sqlite3	12.8.0	Driver SQLite natif Node.js

Justification du choix SQLite :

- Aucun serveur de base de données séparé à administrer
- Fichier unique sur le disque, sauvegarde triviale

- Performances suffisantes pour le volume de données et le nombre d'utilisateurs attendus
- Parfaitement adapté à un hébergement VPS mono-instance

3.3. Modèle de données

La base de données contient les entités suivantes :

Entité	Description
User	Utilisateurs de la plateforme avec rôles (admin / utilisateur)
Document	Métadonnées des fichiers (titre, type MIME, taille, chemin, auteur...)
Folder	Arborescence de dossiers (structure hiérarchique récursive)
Category	Catégories de classement des documents
ActivityLog	Journal des actions utilisateurs (upload, téléchargement, connexion...)

4. Authentification et sécurité

4.1. NextAuth.js

Le système d'authentification est basé sur **NextAuth.js 4.24.13**, une bibliothèque d'authentification éprouvée pour les applications Next.js.

Composant	Version	Rôle
next-auth	4.24.13	Gestion des sessions et de l'authentification
@auth/prisma-adapter	2.11.1	Intégration NextAuth avec Prisma/SQLite
bcryptjs	3.0.3	Hachage sécurisé des mots de passe

4.2. Mécanismes de sécurité

- **Stratégie JWT** : les sessions utilisateurs sont gérées par des tokens JWT avec expiration configurable, sans stockage en base de données
- **Hachage des mots de passe** : bcryptjs applique un hachage bcrypt (coût adaptatif) avant toute persistance en base
- **Contrôle d'accès par rôles** : les routes d'administration (/api/admin/*, upload, suppression) vérifient le rôle de l'utilisateur connecté
- **Validation des fichiers** : type MIME, taille maximale (50 Mo) et chemin d'accès validés avant tout enregistrement
- **Protection contre la traversée de répertoires** : les chemins de fichiers sont systématiquement vérifiés
- **Middleware de protection** : toutes les routes sont protégées au niveau du middleware Next.js

5. Stockage des fichiers

5.1. Principe de stockage

Les fichiers uploadés sont stockés directement sur le **système de fichiers du serveur**, dans un répertoire dédié (`./uploads/`). Cette approche est rendue possible par le choix d'un hébergement VPS (voir section 6), où le stockage est persistant.

Paramètre	Valeur
Répertoire de stockage	<code>./uploads/</code> (configurable via <code>UPLOAD_DIR</code>)
Taille maximale par fichier	50 Mo
Nommage des fichiers	UUID + extension originale (ex. : <code>a3f2c1...pdf</code>)

5.2. Types de fichiers acceptés

- Documents bureautiques : PDF, Word (`.doc`, `.docx`), Excel (`.xls`, `.xlsx`), PowerPoint (`.ppt`, `.pptx`)
- Images : JPEG, PNG, GIF, WebP, SVG, TIFF
- Fichiers texte : texte brut, CSV, HTML, Markdown

6. Infrastructure et hébergement

6.1. Système d'exploitation — Debian 11

Conformément aux recommandations issues de la visioconférence, le système d'exploitation retenu est **Debian 11 « Bullseye »** (GNU/Linux).

Critère	Justification
Stabilité	Debian est reconnue pour ses cycles de release longs et sa grande stabilité en environnement serveur
Sobriété	Distribution légère, sans logiciels superflus, adaptée aux serveurs
Souveraineté	Hébergement sur infrastructure française (cf. choix OVH)
Coût	Distribution libre et gratuite
Maîtrise	Environnement familier pour l'équipe de développement

6.2. Hébergeur — OVH (VPS)

L'hébergement retenu est un **VPS (Virtual Private Server)** chez **OVH**, hébergeur français.

Avantages du VPS OVH :

- **Souveraineté des données** : infrastructure localisée en France, soumise au droit français et européen (RGPD)

- **Coût maîtrisé** : offres à partir de quelques euros par mois, prévisibles et sans facturation à l'usage
- **Stockage persistant** : contrairement aux hébergements *serverless*, le stockage local des fichiers est garanti
- **Contrôle total** : accès SSH, configuration système libre, pas de restriction de plateforme
- **Environnement familier** : OVH est un acteur bien connu et documenté dans l'écosystème francophone

6.3. Architecture serveur

La stack logicielle déployée sur le VPS est la suivante :

Composant	Version recommandée	Rôle
Debian	11 (Bullseye)	Système d'exploitation
Node.js	20 LTS	Runtime JavaScript (serveur)
PM2	dernière stable	Gestionnaire de processus (redémarrage automatique)
Nginx	dernière stable	Reverse proxy, terminaison SSL
Certbot	dernière stable	Certificat SSL/TLS via Let's Encrypt (HTTPS)

6.4. Spécifications serveur recommandées

Ressource	Recommandation
RAM	4 Go minimum
CPU	2 vCPU
Stockage	40 Go SSD (extensible selon le volume documentaire)
Bande passante	Illimitée ou volume suffisant

6.5. Absence de conteneurisation

Conformément aux choix du projet, **aucune conteneurisation Docker** n'est utilisée. Le déploiement est effectué directement sur le système d'exploitation, ce qui simplifie la gestion et réduit la complexité d'exploitation.

7. Outils de développement

7.1. Chaîne de build

Outil	Version	Rôle
TypeScript	5.9.3	Typage statique, compilation vers JavaScript
ESLint	9.39.4	Analyse statique du code (configuration flat config ESLint 9)
PostCSS	—	Transformation CSS (via plugin Tailwind)
tsx	4.21.0	Exécution de scripts TypeScript (ex. : seed de base de données)
Prisma CLI	7.5.0	Génération du client, migrations, studio d'administration

7.2. Scripts disponibles

Commande	Action
<code>npm run dev</code>	Lancement du serveur de développement
<code>npm run build</code>	Compilation de production
<code>npm start</code>	Démarrage du serveur en production
<code>npm run lint</code>	Analyse statique du code
<code>npm run db:migrate</code>	Application des migrations de base de données
<code>npm run db:seed</code>	Initialisation des données de référence
<code>npm run db:studio</code>	Interface d'administration Prisma

7.3. Variables d'environnement

Variable	Description
<code>DATABASE_URL</code>	Chemin vers la base de données SQLite
<code>NEXTAUTH_SECRET</code>	Clé secrète pour la signature des tokens JWT
<code>NEXTAUTH_URL</code>	URL publique de l'application
<code>UPLOAD_DIR</code>	Répertoire de stockage des fichiers (défaut : <code>./uploads</code>)
<code>MAX_FILE_SIZE_MB</code>	Taille maximale d'upload en Mo (défaut : 50)
<code>ANTHROPIC_API_KEY</code>	Clé API pour les fonctionnalités IA (phase 2)

8. Perspectives — Phase 2 : Intelligence Artificielle

8.1. Intégration IA existante

La plateforme intègre d'ores et déjà le SDK **Anthropic Claude** (`@anthropic-ai/sdk v0.80.0`) pour des fonctionnalités de recherche sémantique et d'analyse documentaire via l'API Claude.

8.2. Considérations sur les coûts d'infrastructure IA

Comme évoqué lors de la visioconférence, le coût d'une infrastructure IA peut rapidement devenir significatif. Deux options sont à étudier :

Option	Avantages	Inconvénients
API cloud (Mistral, Claude, Azure OpenAI...)	Pas d'infrastructure à gérer, montée en charge automatique	Coût variable, dépendance à un tiers, données transmises hors du périmètre
Auto-hébergement (modèles open source)	Maîtrise des données, coût fixe	Infrastructure GPU requise, coût initial élevé, expertise nécessaire

Recommandation : une analyse de coût et un chiffrage précis seront réalisés par la Junior Conseil UTT en amont de la phase 2, en fonction des volumes de traitement attendus et des contraintes de confidentialité des données.

9. Synthèse

Composant	Technologie / Version	Statut
Framework web	Next.js 16.2.1	En production
Front-end	React 19.2.4 + TypeScript 5.9.3	En production
UI	Radix UI + Tailwind CSS 4.2.2	En production
Base de données	SQLite via Prisma 7.5.0	En production
Authentification	NextAuth 4.24.13 + bcryptjs	En production
Stockage fichiers	Système de fichiers local (./uploads/)	En production
IA	Anthropic Claude SDK 0.80.0	Intégré (phase 2)
OS serveur	Debian 11 « Bullseye »	Retenu
Hébergeur	OVH — VPS	Retenu
Reverse proxy	Nginx + Certbot (HTTPS)	À déployer
Gestionnaire de processus	PM2	À déployer
Conteneurisation	Aucune (Docker non utilisé)	—